

LA IA COMO INFRAESTRUCTURA, EL HUMANO COMO AUTOR

Una Ingeniería de Gestión de Software de IA para el trabajo con IA

Pelayo Urzaiz

Abril 2026

Prefacio

El desarrollo de software con inteligencia artificial se rige hoy por un modo de trabajo híbrido que la industria ha asentado en los últimos dos años: **trabajar con la IA**. Ingeniero y modelo operan colaborativamente sin un reparto fijo de tareas —a veces escribe el humano y la IA revisa o refactoriza, a veces produce la IA y el humano valida e integra, a veces ambos dialogan sobre una decisión—. Es lo que efectivamente hacen hoy los equipos que han integrado la IA en su día a día, y es la posición desde la cual se discute todo lo demás.

En el desarrollo de software industrial se utiliza hoy una disciplina consolidada para organizar la producción con rigor: la **Ingeniería de Gestión de Software** (SEM, *Software Engineering Management*) —roles separados, procesos formales, artefactos persistentes y herramientas de coordinación que convierten el rigor en efecto estructural del sistema en lugar de virtud individual—. La SEM clásica contempla las dimensiones que todo desarrollo riguroso debe verificar: el **Arquitecto** revisa que no se salga de la arquitectura, el **Product Owner** que se ajuste al alcance y al fit con el producto, el **Quality Assurance** que la cobertura sea suficiente, el **Security Officer** que no entren vulnerabilidades.

La irrupción de la IA en el flujo de trabajo introduce, sin embargo, un problema operativo que esa SEM no absorbe: **revisar lo que la IA produce es caro**. La IA, por deficiencias estructurales —olvido entre sesiones, ausencia de perspectiva propia, incapacidad de delimitar alcance, sesgo confirmatorio—, hace continuamente cosas que no debería hacer: se sale de la arquitectura acordada, introduce código redundante, toca módulos fuera del alcance, olvida convenciones, contradice decisiones ya tomadas —transgrede, precisamente, las mismas dimensiones que la SEM clásica separa en roles—, y el desarrollador tiene que verificarlas todas en paralelo mientras produce. Los roles superiores, por su parte, fueron pensados para revisar a un humano que **escribe** código —que al escribir ya internaliza convenciones, arquitectura y alcance—, no a un humano que a su vez **revisa** output de IA. El coste, entonces, no se absorbe arriba sino que **se multiplica en capas**: recae primero sobre el desarrollador, que debe verificarlo todo en paralelo, y después íntegro otra vez sobre los roles superiores, que no pueden confiar en que esa revisión multitarea haya cubierto su dimensión y la hacen desde cero. Reportes recientes lo confirman: el ensayo controlado de METR (2025)¹ mide una **ralentización efectiva del 19 %** al trabajar con IA en repositorios maduros;

¹METR. (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. arXiv:2507.09089. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>

Uplevel (2024)² documenta un **aumento del 41 %** en la tasa de bugs por commit entre usuarios de Copilot; y la Stack Overflow Developer Survey (2025)³ recoge que el **66 % de los desarrolladores reporta frustración con respuestas “casi correctas pero no del todo”** que exigen depuración costosa.

No es de extrañar, frente a ese coste multiplicado, que en la práctica emerjan dos reacciones opuestas. Unos **relegan la IA a tareas accesorias —tests, andamiaje, documentación— o la dejan de usar**: evitan el coste de revisar, pero pagan el precio en **pérdida de productividad** y en quedar fuera del ecosistema productivo que se está consolidando. Otros **dejan de revisar y se deslizan al *vibe coding*** en el sentido que popularizó Karpathy (2025)⁴: ahorran el coste de revisar en el momento, pero dejan pasar al proyecto todo lo que la revisión atenta antes atrapaba —errores sutiles, vulnerabilidades, decisiones arquitectónicas erradas, código inmantenible—, convirtiendo el output en una **bomba de relojería** que estalla después como deuda cognitiva, bugs de producción y sistemas que nadie puede evolucionar.

La tesis de este trabajo es que **el trabajo con IA requiere su propia infraestructura de gestión**: una **Ingeniería de Gestión de Software de IA** — reformulación de la SEM clásica materializada como capa de agentes de IA que rodea el trabajo del ingeniero, sin sustituir a ninguno de los roles humanos. Cada rol que la SEM clásica reconoce (Product Owner, Arquitecto, Quant, Desarrollador, Quality Assurance, Security, DevOps, Coach) recibe su propio **agente homólogo**: un interlocutor de IA especializado con el que ese rol dialoga cuando lo está ejerciendo, que conoce su dominio, lee y escribe la parte del vault que le concierne y lo acompaña en su trabajo. En equipo, cada humano tiene su agente. En trabajo individual, el mismo humano alterna entre agentes según qué rol esté ejerciendo en cada fase. Estos agentes leen, escriben y verifican sobre el vault, coordinan handoffs **y pueden generar código bajo la dirección del humano que ejerce el rol**; lo que no hacen es apropiarse de la ejecución: la **autoría y el juicio técnico permanecen en el humano**, que dirige el proceso, conoce lo producido y firma. La IA no ocupa el lugar del humano; ocupa el lugar que antes ocupaba la infraestructura organizacional alrededor del trabajo humano, reformulada ahora para absorber también el coste específico de trabajar con IA.

La clave es una **memoria compartida entre humanos y agentes**: un vault estructurado, segmentado por rol, del que tanto los humanos como sus agentes homólogos leen y escriben —ADRs, plan de fase, alcance, convenciones, handoffs, verificaciones—. Sobre ella operan tres mecanismos: **segmentación por perspectiva** (cada agente ve y escribe solo su fragmento, eliminando la dispersión y el desvío de alcance), **disciplina reactiva** (los agentes de los roles superiores —Arquitecto, PO, QA, Security, Coach, DevOps— intervienen automáticamente durante la escritura, al hacer commit y al abrir PR, cada uno sobre su dimensión), y **trazabilidad atómica** (cada fase vive en una rama git que versiona a la vez código y artefactos de gestión). Esa misma memoria se proyecta además como **capa visual unificada** —dashboards sobre estado de features y actividad de agentes, tableros de flujos de trabajo, diagramas vinculados a ADRs, vistas comparativas de experimentos—: el ingeniero no invoca el sistema, lo **habita**. Lo que disuelve el coste multiplicado del inicio es que los agentes verifican **contra la misma fuente de verdad** que los humanos usarán después: el Architect-Agent lee el mismo ADR que el Arquitecto humano leerá en el PR, el PO-Agent aplica la misma definición de alcance que el PO humano validará al cierre. La revisión humana deja de ser *descubrimiento* —buscar qué se desvió— y pasa a ser *validación* —confirmar lo ya registrado como

²Uplevel Data Labs. (2024). *Can Generative AI Really Accelerate Software Development?* <https://resources.uplevelteam.com/gen-ai-for-coding>

³Stack Overflow. (2025). *2025 Developer Survey — AI Section*. <https://survey.stackoverflow.co/2025/ai>

⁴Karpathy, A. (2 de febrero de 2025). *There’s a new kind of coding I call “vibe coding”* [Publicación en X]. <https://x.com/karpathy/status/1886192184808149383>

conforme—.

Resuelta la raíz, las dos reacciones del inicio pierden su motivo. El **escepticismo** pierde el suyo: la revisión deja de ser el cuello de botella que hacía más rentable escribir a mano, porque la infraestructura la absorbe como efecto estructural del sistema en lugar de trabajo discrecional del practicante. El *vibe coding* pierde el suyo: la infraestructura impide delegar sin leer, sostiene la revisión aunque el practicante afloje, y deja rastro de cada decisión. Ninguna de las dos resulta atractiva cuando el coste que las motivaba ha sido ya absorbido estructuralmente.

Este marco aplica a cualquier ingeniero que trabaje con IA, sea en equipo o en solitario. Como la infraestructura **es, por construcción, una materialización de SEM** —cada rol tiene un agente homólogo espejo que lo sujeta en su dimensión—, lo que antes exigía repartir roles entre varias personas para producir con rigor puede ahora sostenerlo un solo humano sin colapsar. La respuesta al avance de la IA no es reasignar al **humano** a tareas que la IA no hace, sino reasignar a la **IA** para que el humano pueda seguir siendo el **autor** de lo que produce, apoyado en una infraestructura que lo sujeta en cada rol. **Juzgar, decidir y firmar no sustituye ejecutar: lo enmarca.**